Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Curso: Introducción a la Programación LL

Proyecto 1 – Juego 2D de Plataformas

BrickBound

Esteban Alejandro Sánchez Ledezma

Jeff Schmidt Peralta

2026

Índice

Objetivo del Proyecto **3**

Elementos esenciales del nivel 3

Planteamiento General **4**

Validación y Creación de Mapas **6**

Funciones Principales **8**

Problemas Encontrados y Solución Aplicada **10**

Conclusión **12**

Uso de la IA **13**

IA´s utilizadas 15

Referencias **16**

Objetivo del Proyecto

BrickBound es un videojuego 2D de plataformas desarrollado en Python usando la biblioteca Tkinter. El objetivo principal del juego es que el jugador lleve a su personaje desde el punto de inicio del mapa hasta la meta final, superando los obstáculos del camino.

Para ganar, el jugador debe desplazarse horizontalmente, aprovechar las escaleras para subir o bajar entre plataformas, y evitar o sobrevivir el contacto con los dos tipos de enemigos y las trampas presentes en el escenario. La partida termina en victoria al llegar a la meta y en derrota al quedarse sin las tres vidas disponibles.

Elementos esenciales del nivel

- **Plataformas (bloques):** superficies sólidas sobre las que el jugador puede caminar y saltar.
- **Escaleras:** permiten al jugador moverse verticalmente entre plataformas.
- **Enemigo patrulla:** se mueve horizontalmente sobre las plataformas y revierte su dirección al llegar al borde.
- **Enemigo lanzador:** permanece estático y dispara proyectiles en línea recta a intervalos regulares.
- **Trampas:** al contacto quitan una vida y regresan al jugador al punto de inicio.
- **Punto de inicio:** posición donde aparece el jugador al comenzar o al reiniciar tras una trampa.
- **Meta:** destino final cuyo alcance determina la victoria de la partida.

Planteamiento General

Representación del escenario

El mapa se representa como una matriz de 15 filas por 25 columnas, donde cada celda contiene un número entero que indica qué elemento ocupa ese espacio. Los códigos usados son: 0 = vacío, 1 = bloque, 2 = escalera, 3 = trampa, 4 = enemigo patrulla, 5 = enemigo lanzador, 6 = inicio del jugador y 7 = meta. Esta representación permite cargar, guardar y validar mapas de forma sencilla con un archivo de texto.

Control del movimiento del jugador

El jugador se controla con las teclas de dirección o WASD. En cada frame del loop del juego se revisa el diccionario de teclas presionadas y se ajusta la velocidad horizontal. Si el jugador está sobre una escalera, se desactiva la gravedad y se permite movimiento vertical libre. Fuera de la escalera, la gravedad se aplica sumando una constante a la velocidad vertical en cada frame.

Sistema de salto

El salto solo es posible cuando el jugador está en contacto con el suelo (*en_suelo = True*). Al presionar espacio o la flecha arriba, la velocidad vertical se establece en un valor negativo que lo impulsa hacia arriba. La gravedad hace que esta velocidad aumente gradualmente hasta que el jugador aterriza en una plataforma, momento en que *en_suelo* vuelve a ser *True*.

Comportamiento de los enemigos

El enemigo patrulla se mueve horizontalmente a velocidad constante. En cada frame se verifica si hay una pared adelante o si el piso desaparece; en cualquiera de esos casos la dirección se invierte. El enemigo lanzador es estático y dispara un proyectil hacia la izquierda cada 120 frames aproximadamente. Los proyectiles se eliminan al salir del mapa o chocar con un bloque.

Verificación de la llegada a la meta

En cada frame se calcula en qué celda de la matriz se encuentra el centro del jugador. Si el valor de esa celda es 7 (META), la función de colisiones devuelve 'victoria', el loop del juego se detiene, se guarda el puntaje y se muestra la pantalla de resultado.

Validación y Creación de Mapas

El juego incluye un mapa predeterminado diseñado por el programador y un editor que permite al jugador crear mapas adicionales de forma manual.

Mapa predeterminado

El mapa 'The Dungeon' está codificado directamente en el programa como una lista de listas (matriz 15x25). Esto garantiza que siempre esté disponible sin depender de ningún archivo externo. Su diseño coloca al jugador en la esquina inferior izquierda y la meta en la esquina superior derecha, conectados por una serie de plataformas y escaleras que forman un camino válido y completable además de integrar los distintos enemigos que incluye el juego.

Editor de mapas

El editor permite al jugador diseñar un mapa manualmente sobre una cuadrícula de 25 x 15 celdas. Cada celda se pinta con el elemento seleccionado en el panel lateral. Los mapas creados se guardan en el archivo mapas.txt con el siguiente formato:

- Primera línea: NOMBRE: seguido del nombre del mapa y autor (opcional).
- Siguientes 15 líneas: cada fila de la matriz con los valores separados por comas.
- Línea separadora --- para delimitar mapas dentro del mismo archivo.

Validación

Antes de guardar un mapa, el sistema verifica que contenga exactamente un punto de inicio (6) y una meta (7). Si alguna de estas condiciones no se cumple, se muestra un mensaje de error y no se permite guardar ni iniciar una partida con ese mapa.

El puntaje base del mapa se calcula sumando puntos por enemigos y trampas (que aumentan la dificultad) y restando puntos por bloques y escaleras (que facilitan el desplazamiento), con un mínimo garantizado de 100 puntos.

Funciones Principales

loop_juego()

Es el motor central del juego. Se llama a sí misma cada 16 milisegundos usando ventana.after(), lo que produce aproximadamente 60 frames por segundo. En cada llamada actualiza la posición del jugador, mueve los enemigos, desplaza los proyectiles, verifica todas las colisiones y redibuja el frame completo en el canvas. Es relevante porque sin este ciclo el juego sería estático; toda la animación y lógica dependen de que esta función se ejecute de forma continua y ordenada.

verificar_colisiones_jugador()

Detecta si el jugador tocó una trampa, un enemigo, un proyectil o la meta en el frame actual. Para las trampas usa la posición en la matriz; para enemigos y proyectiles usa comparación de rectángulos (bounding box). Aplica el sistema de invencibilidad temporal tras recibir daño para que el jugador no pierda varias vidas en un solo contacto. Es relevante porque centraliza toda la lógica de consecuencias del juego: victoria, derrota, pérdida de vida y reinicio en el punto de inicio.

cargar_sprites()

Carga todos los assets visuales del juego desde la carpeta assets usando la biblioteca Pillow. Extrae frames individuales de los spritesheets con PIL.Image.crop(), genera versiones espejo con transpose(FLIP_LEFT_RIGHT) para cuando el jugador y los enemigos miran a la izquierda, y crea el efecto de daño blanqueando todos los píxeles del sprite del jugador. Es relevante porque centraliza la preparación visual del juego y permite que el resto del código use simplemente sprites['nombre'] sin preocuparse por el procesamiento de imágenes.

guardar_mapa_archivo()

Lee todos los mapas existentes en mapas.txt, verifica si ya existe uno con el mismo nombre para reemplazarlo o lo agrega como nuevo, y escribe la lista completa de vuelta al archivo. Convierte cada fila de la matriz de enteros a una línea de texto con valores separados por comas. Es relevante porque garantiza que los mapas persistan entre sesiones y que no se pierdan mapas previos al guardar uno nuevo.

leer_puntajes() y guardar_puntaje()

leer_puntajes() abre puntajes.txt, convierte cada línea en una lista [nombre, puntaje] y ordena el resultado de mayor a menor usando el algoritmo de burbuja. *guardar_puntaje()* agrega el nuevo resultado, reordena y conserva solo los 5 mejores antes de escribir el archivo. Son relevantes porque implementan la persistencia del ranking entre sesiones sin depender de ninguna librería externa para el manejo de datos.

Problemas Encontrados y Solución Aplicada

Durante la creación del juego se encontraron múltiples errores y fallos que impedían la correcta ejecución del juego, en esta sección encontrará los más relevantes y difíciles de corregir ya que requierían que analizará cálculos y funciones, investigara soluciones e incluso implementar soluciones basadas en respuestas generadas por inteligencia artificial.

Sprites de enemigos atravesaban las plataformas

El enemigo patrulla usaba su coordenada "Y" dividida por el tamaño de celda (32px) para calcular en qué fila estaba. Sin embargo, el sprite del enemigo mide 48px, por lo que los pies reales del enemigo estaban 16px más abajo de lo que el código calculaba. Esto hacía que la detección de piso fallara y el enemigo quedará dentro del bloque. La solución fue usar Y + 48 para calcular la fila de los pies, y ajustar el offset de posición inicial con -16px para que los sprites quedarán visualmente sobre las plataformas.

Jugador no podía mirar hacia la izquierda

Los sprites del personaje siempre miraban hacia la derecha, lo que hacía que el movimiento hacia la izquierda se viera incorrecto. La solución fue generar versiones espejo de cada frame al momento de cargar los sprites, usando *PIL.Image.transpose(FLIP_LEFT_RIGHT)*. Se crearon listas paralelas con sufijo *_e (espejo)* y la función *obtener_sprite_jugador()* elige entre la lista normal o la espejo según la dirección actual del jugador.

Puntajes y mapas no se encontraban al ejecutar el programa

Python buscaba los archivos puntajes.txt y mapas.txt en el directorio de trabajo actual, que en Windows no siempre coincide con la carpeta donde está el .py. La solución fue usar os.path.dirname(os.path.abspath(__file__)) para construir la ruta absoluta al archivo basándose

en la ubicación del script, garantizando que siempre se encuentre sin importar desde dónde se ejecute Python.

Botón de volver no aparecía en el editor de mapas

El panel lateral del editor tiene altura fija y *pack_propagate(False)*, por lo que los widgets que no cabían simplemente no se mostraban. La solución fue crear un frame separado con *side='bottom'* para los botones de acción (guardar, limpiar, volver), anclándolos al fondo del panel independientemente de cuánto contenido haya arriba.

Hitbox de enemigos demasiado grande

Los sprites de 48px tienen zona transparente alrededor del personaje real, lo que hacía que el jugador colisionara antes de visualmente tocar al enemigo. La solución fue ajustar la comparación de rectángulos en *verificar_colisiones_jugador()* para usar offsets internos (píxeles 8-40 en X y 16-48 en Y) que reflejan el área real del personaje dentro del sprite.

Conclusión

El proyecto BrickBound logró implementar un juego 2D de plataformas completamente funcional en Python con Tkinter, cumpliendo todos los requisitos del enunciado: movimiento del jugador, salto, escaleras, dos tipos de enemigos funcionales, editor de mapas con validación, sistema de puntajes persistente en archivo de texto, música con control de reproducción y flujo completo entre pantallas.

Lo más importante aprendido durante el desarrollo fue cómo estructurar un programa grande sin el uso de clases, delegando la organización a funciones bien definidas con un propósito claro cada una, y usando diccionarios para agrupar el estado de entidades como el jugador y los enemigos. También fue valioso entender cómo funciona un game loop básico y cómo el método after() de Tkinter permite simularlo.

Los aspectos más retadores fueron el sistema de colisiones, especialmente ajustar las hitboxes para que coincidieran con los sprites reales y no con sus rectángulos completos, y la compatibilidad de rutas de archivos entre sistemas operativos. El proyecto dejó una comprensión sólida de cómo estructurar programas complejos con los fundamentos básicos de la programación.

Uso de la IA

Durante la creación del proyecto se utilizó la inteligencia artificial como una herramienta de apoyo tanto para facilitar determinadas tareas como para resolver problemas complejos para los cuales no se encontraba una solución clara o conocida. Cabe destacar que su uso se limitó únicamente a los momentos en los que fue realmente necesaria, con el fin de evitar interrumpir el proceso de aprendizaje adquirido durante el desarrollo del proyecto.

Además, cada solución o ayuda proporcionada por la IA fue analizada rigurosamente para comprender su funcionamiento y aprender de ella. Esto permitió adquirir nuevos métodos y enfoques ingeniosos que, debido al estilo de pensamiento o experiencia del desarrollador, probablemente no se habrían considerado de manera inmediata.

En esta sección se recopilan principalmente los problemas en los que la IA contribuyó directamente a la resolución o mejora del proyecto. Otras ayudas recibidas no fueron incluidas debido a su menor relevancia, ya que consistieron únicamente en consultas relacionadas con organización de datos, selección de colores, fórmulas, análisis de información, mejoras de redacción, mejoras de diseño y detección de errores o problemas.

Diseño del sistema de salto

Pront:

“Necesito implementar el salto del jugador en un juego 2D con Tkinter. El personaje tiene velocidad vertical y gravedad que se aplican cada frame. ¿Cómo manejarías el salto para que solo sea posible cuando el jugador está en el suelo, y cómo evitarías que salte en el aire?”

IA:

“Para manejar el salto de forma correcta, la clave es tener una variable booleana en_suelo que indique si el jugador está apoyado sobre una superficie. El salto solo se activa si en_suelo es

True: en ese momento se asigna un valor negativo a vel_y (hacia arriba) y se marca en_suelo como False. La gravedad suma una constante a vel_y en cada frame, por lo que el jugador sube, frena y cae naturalmente. Cuando la función de colisión vertical detecta contacto con un bloque abajo, se corrige la posición y se vuelve a poner en_suelo en True, habilitando el siguiente salto.”

Problema con rutas de archivos en Windows

Pront:

“Los archivos puntajes.txt y mapas.txt no se encuentran cuando ejecuto el programa desde VSCode. Sé que están en la misma carpeta que el .py pero Python no los encuentra. El os.path.exists() devuelve False aunque el archivo sí existe.”

IA:

“El problema es que Python usa el directorio de trabajo actual (cwd), que en Windows con VSCode no siempre es la carpeta del script. La solución es construir la ruta absoluta basada en la ubicación del propio archivo .py usando: os.path.join(os.path.dirname(os.path.abspath(__file__)), 'puntajes.txt'). Esto garantiza que sin importar desde dónde se ejecute Python, siempre encuentre el archivo relativo al script.”

Carga de sprites y animaciones desde spritesheets

Pront:

“Tengo los sprites del personaje en un spritesheet de 384x32px con 12 frames de 32x32 cada uno. ¿Cómo los cargo en Tkinter usando Pillow y cómo hago que el personaje mire a la izquierda sin tener imágenes separadas para esa dirección?”

IA:

“Con Pillow podés extraer cada frame usando img.crop((frame_idx * 32, 0, frame_idx * 32 + 32, 32)) y convertirlo a PhotoImage con ImageTk.PhotoImage(frame). Para la dirección, no necesitás imágenes separadas: usá frame.transpose(Image.FLIP_LEFT_RIGHT) para generar la versión espejo en memoria al momento de cargar. Guardás ambas versiones en listas paralelas (jugador_run y jugador_run_e) y elegís cuál usar según la dirección actual del jugador.”

Persistencia de múltiples mapas en un archivo de texto

Pront:

“Quiero guardar varios mapas en un solo archivo .txt de forma que pueda leerlos de vuelta como matrices. Cada mapa es una lista de 15 listas con 25 enteros. ¿Qué formato usarías para que sea simple de escribir y leer sin librerías externas?”

IA:

“Un formato sencillo y legible: cada mapa empieza con una línea NOMBRE:nombre_del_mapa, seguida de 15 líneas donde cada fila de la matriz se escribe con los valores separados por comas (0,1,1,0,...), y al final una línea --- como separador entre mapas. Al leer, detectás el inicio con startswith('NOMBRE:'), acumulás filas hasta el --- y convertís cada línea con linea.split(',') pasando cada valor a int(). Así podés guardar y cargar cualquier cantidad de mapas sin depender de JSON ni otras librerías.”

IA´s utilizadas

Se utilizó principalmente Cloud AI por su increíble capacidad para manejar código, seguido de Gemini ya que excelente para encontrar información y por último Chat GPT para la mejora de Textos.

Referencias

Anthropic. (n.d.). *Claude (claude-sonnet-4-20250514) [Modelo de lenguaje de gran escala]*. Home \ Anthropic. Retrieved 2026, from https://www.anthropic.com

Barros, E. (2025|). *TKINTER DESDE CERO*. Youtube. Retrieved 2026, from https://www.youtube.com/playlist?list=PLVzwufPir357GruwLwGXCi_PXrbsASqEvhttps://www.youtube.com/playlist?list=PLVzwufPir357GruwLwGXCi_PXrbsASqEv

Código Espinoza. (2025). *Crea un Juego de Supervivencia en Python*. Youtube. Retrieved 2026, from https://www.youtube.com/playlist?list=PL2ImQBOumhlNs_4qVI578JJ5eXaMdTCm9

Código Espinoza - Automatiza tu Vida. (2023). *Curso de Tkinter: Crea Interfaces Gráficas en Python Desde Cero*. Youtube. Retrieved 2026, from http://youtube.com/playlist?list=PL7HAy5R0ehQXb2aFKOKyeCMequxyb5jzJ

CraftPix.Net. (2024, Marzo 20). *Free Simple Platformer Game Kit Pixel Art*. CraftPix.Net. Retrieved 2026, from https://craftpix.net/freebies/free-simple-platformer-game-kit-pixel-art/?num=1&count=288&sq=simple%20plataform&pos=4

DeepSeek. (2025). *DeepSeek*. DeepSeek [Large language model]. https://chat.deepseek.com/

Developeando. (2022, Julio 22). *¿Cómo usar GitHub? - ¡Todo lo que necesitás saber!* Youtube. https://www.youtube.com/watch?v=44ziZ12rJwU&t=53s&pp=ygUWZ2l0aHViIGJhc2ljbyBlc3Bhw7FvbNIHCQneCgGHKiGM7w%3D%3D

Fagbuyiro, D., & Hernandes, R. (2024, Octubre 17). *Manejo de archivos en Python: Cómo crear, leer y escribir en un archivo*. FreeCodeCamp.org. Retrieved 2026, from

https://www.freecodecamp.org/espanol/news/manejo-de-archivos-en-python-como-crear-leer-y-escribir-en-un-archivo/

Google. (2026). *Gemini [Modelo de lenguaje de IA]*. Google Gemini. https://gemini.google.com/

Kajiru. (2025, Diciembre 23). *Getting Started with 2D Games Using Tkinter (Part 4): Creating Sprites*. DEV. Retrieved 2026, from https://dev.to/sdkfz181tiger/getting-started-with-2d-games-using-tkinter-part-4-creating-sprites-52jl

Lousberg, K. (2024). *Top free game assets - itch.io*. Itch.io. Retrieved 2026, from https://itch.io/game-assets/free

OpenAI. (2026). *ChatGPT (versión GPT-5.5)*. ChatGPT[Modelo de lenguaje grande]. https://chatgpt.com

Piensa 3D. (2017). *https://www.youtube.com/watch?v=h5hW3ynHvhM*. Youtube. Retrieved 2026, from https://www.youtube.com/watch?v=h5hW3ynHvhM

Pillow Contributors. (2024). *Pillow*. (PIL Fork) documentation. Retrieved 2026, from https://pillow.readthedocs.io/en/stable/

PixelLab. (n.d.). *PixelLab - AI Generator for Pixel Art Game Assets*. PixelLab Create unique pixel art assets for your game 10x faster with AI. Retrieved 2026, from https://www.pixellab.ai/

Pygame. (2024). *pygame.mixer — pygame v2.6.0 documentation*. Pygame. Retrieved 2026, from https://www.pygame.org/docs/ref/mixer.html

Python Software Foundation. (2024). *os.path — Common pathname manipulations — Python 3.14.5 documentation*. Python documentation. https://docs.python.org/3/library/os.path.html

Python Software Foundation. (2024). *tkinter — Python interface to Tcl/Tk — Python 3.14.5 documentation*. Python documentation. Retrieved 2026, from https://docs.python.org/3/library/tkinter.html

Real Python. (n.d.). *Python Game Development: Tutorials & Code Examples – Real Python*. Real Python. https://realpython.com/tutorials/gamedev/

Nota: Los assets de audio y sprites utilizados fueron obtenidos de fuentes de libre uso y se referencian según las condiciones de sus licencias originales.